

ANS Forth Test Suite, Organized by forth6809.asm

Section

Source: <https://forth-standard.org/standard/testsuite> (Annex F), fetched 2026-07-15.

Explicitly redistributable per the test harness's own header: "(C) 1995 JOHNS HOPKINS UNIVERSITY / APPLIED PHYSICS LABORATORY - MAY BE DISTRIBUTED FREELY AS LONG AS THIS COPYRIGHT NOTICE REMAINS."

What's here

- `00a_tool_ext_conditionals.fs` — `[IF] / [ELSE] / [THEN]` (Programming- Tools word set), which forth6809.asm does not implement natively. `ttester.fs` needs them immediately, in its own `HAS-FLOATING / HAS-FLOATING-STACK` checks, so this file loads first, ahead of even `ttester.fs`. It's the informative reference implementation given by the standard itself (forth-standard.org/standard/tools/BracketELSE), built only from words forth6809.asm already has (`BL WORD COUNT S" COMPARE REFILL IF/ELSE/THEN BEGIN/WHILE/REPEAT/UNTIL ?DUP EXIT IMMEDIATE`) — nothing further needs to be assembled into the ROM to run the ANS suite.
- `ttester.fs` — the test harness itself (ANS Forth Standard, Annex F, Section F.2.3), reorganized here into its own file rather than left bundled inline with anything else. Defines `T{ / -> / }T` and every supporting word (`ERROR`, `EMPTY-STACK`, the `HAS-FLOATING / HAS-FLOATING-STACK` detection, and the `X}T / R}T`-family closing words for mixed cell/float stack pictures). Load this first, before the prelude or any section file. Verified after transcription: `[IF]` and `[THEN]` counts match exactly (8/8), and `:` `/` ; counts match exactly (63/63) — no unbalanced conditional-compilation or colon definition introduced while extracting it from the fetched page.
- `00_test_prelude.fs` — shared setup every section depends on: the basic two's-complement assumptions (`BITSSET?`), the bound constants (`MAX-UINT`, `MAX-INT`, `MIN-INT`, `MID-UINT`, `MID-UINT+1`, `MSB`), the boolean constants (`0S/1S` via `<FALSE>/<TRUE>`), the floored-vs-symmetric division helpers (`IFFLOORED/IFSYM`), the string-compare helper (`S=`), and the shared memory buffers used by the `FILL/MOVE` tests (`FBUF/SBUF/SEEBUF`). Load this after `ttester.fs`, before any section file.
- `NN_name.tests.fs` — one file per forth6809.asm section (matching the exact same numbering and filenames as `forth6809_split/`), containing every ANS test block for

the words that section implements.

153 test blocks placed across 17 section files (sections that implement no words with an official ANS test — e.g. section 5's inner-interpreter primitives like LIT/BRANCH, which have no directly-callable ANS name — have no file, since there was nothing to place there).

Load order and independence — now resolved

The original ANS suite is one continuous stream where later tests reuse words and constants defined by earlier ones (explicit in the standard's own F.3 narrative: "these are included in the appropriate test"). Splitting it by forth6809.asm section preserved the *word groupings* but broke that original ordering — this has since been fixed so every section file is independently runnable, in any order, any number of times, from a single shared harness+prelude load:

- `09_outer_interpreter.tests.fs` (FIND) used to depend on `GT1 / GT2` from `13_compiling_words.tests.fs`. It now carries its own copy of both (`: GT1 123 ; : GT2 ['] GT1 ; IMMEDIATE`), so section 13 no longer needs to run first.
- Every section file now opens with `MARKER Mnn` (nn = the section number, e.g. `M08`) right after its header comment, and closes with a bare invocation of `Mnn` as its last line.
`MARKER snapshots DPHERE / CODEHERE / VARHERE / LATEST` when created and restores all four when invoked (confirmed directly against forth6809.asm's `MARKERW / DOMARKER`), so running a section file and then letting it invoke its own marker leaves the dictionary, value-space and search chain exactly as they were right after `ttester.fs + 00_test_prelude.fs` were loaded — no leftover state for the next section file to trip over, whatever order they run in.
- `18_memory.tests.fs` 's `MOVE` test still depends on the `FBUF / SBUF / SEEBUF` state left behind by the immediately preceding `FILL` test — this is fine, since both are in the same file and thus inside the same `MARKER` bracket.
- Several `20_numeric_output.tests.fs` tests use `MAX-BASE` , `#BITS-UD` , and `S=` , all provided by `00_test_prelude.fs` , which sits outside every section's own `MARKER` bracket and is loaded once, up front.

The fixed load order is now simply: `00a_tool_ext_conditionals.fs` , `ttester.fs` , `00_test_prelude.fs` , then any subset of section files, in any sequence — see `ans_test_runner.py` (one level up), which automates exactly this against either MAME or a real serial-connected MECB6809.

Two real gaps this exercise surfaced — one still open, one since

resolved

- **TRUE and FALSE were not implemented as dictionary words** when this test suite was first organized — checked directly against `forth6809.asm` 's dictionary section at the time: only the internal assembler constants `TRUEV` / `FALSEV` existed, never exposed as `CONSTANT TRUE` / `CONSTANT FALSE` a running program could call. This has since been resolved: both are now real dictionary words (`CONSTANT TRUE -1` / `CONSTANT FALSE 0`), and their tests (`F.6.2.1485 FALSE` , `F.6.2.2298 TRUE`) are now included at the end of `26_abort_quit_headers.tests.fs` , alongside that section's other ROM-resident, hand-built words.
- Everything else in Core (F.6.1) and the Core Extension subset this system implements (F.6.2) mapped cleanly to exactly one section.
- **Transcription bug, since fixed:** several "See F.x.x.xxxx WORD." cross-reference notes (and one "The following tests..." narrative line, in `08_defining_words.tests.fs`) had lost their leading `\` comment marker when the original web page's text was extracted — each would have been fed to the interpreter as literal, undefined words ("See", "F.6.1.0450", "The", "following", ...) the moment that file loaded. Found and fixed across all 17 section files while wiring up `ans_test_runner.py` .

Running the suite

`ans_test_runner.py` (one directory up from here) automates loading all of this into a live `forth6809` image and reporting pass/fail per section. It talks to either a MAME instance (spawned the same way `run_all_tests.sh` does, over a `-rs232 null_modem -bitb socket.host:port` TCP connection) or a real serial-connected MECB6809 with the Forth binary in EPROM (via `pyserial`). Rather than pasting text at a fixed baud rate and hoping delays are long enough — the approach currently being fought with `minicom` — it sends `forth6809`'s *own* line-by-line "ok"/ "ERROR" reply as the pacing signal: one line goes out, the runner blocks until that line's own reply comes back, then the next line goes out. Long test-file lines (some exceed 1000 characters) are automatically re-wrapped to fit under `TIBBUFL` (80 bytes) without ever splitting inside a `s" / ." / \` construct. See the script's own `--help` and module docstring for every option.

Not included

Double-Number, Facility, File-Access, Floating-Point, Memory-Allocation, and Search-Order word-set tests are all omitted — `forth6809` implements none of those word sets (Search-Order's absence is documented explicitly in the ClaudeForth documentation, Section 4.5).

Most of the Programming- Tools word set is also omitted (AHEAD, CS-PICK, CS-ROLL, N>R, [THEN], etc. have no ANS test coverage relevant here since this system's own Tools word set — `.S` , `WORDS` , `DUMP` — are non-standard extensions with no official ANS test cases to begin with).